Лабораторная №2 на шаблоне.

Исходный код:

```
#include <iostream>
#include <vector>
#include <set>
#include <tuple>
#include <algorithm>
#include <fstream>
#include <cstdlib>

using namespace std;

using Vertex = int;
using Cost = int;

struct Edge {
    Vertex from;
    Vertex to;
    Cost cost;

    bool operator<(const Edge& rhs) const {
        return tie(from, to, cost) < tie(rhs.from, rhs.to, rhs.cost);
    }
};

using Edges = vector<Edge>;

// Система непересекающихся множеств (DSU).
class DisjoinSets {
    vector<Vertex> parent_;
    vector<Vertex> rank_;

protected:
    void Link(const Vertex& u, const Vertex& v) {
        if (rank_[u] > rank_[v]) {
            parent_[v] = u;
        } else {
            parent_[u] = v;
            if (rank_[u] == rank_[v])
                ++rank_[v];
        }
    }

public:
    explicit DisjoinSets(const Vertex& num)
        : parent_(num), rank_(num, 0) {
        for (Vertex v = 0; v < num; ++v)
            MakeSet(v);
```

```
    }

    void MakeSet(const Vertex& v) {
        parent_[v] = v;
        rank_[v] = 0;
    }

    const Vertex& FindSet(const Vertex& v) {
        if (parent_[v] != v)
            parent_[v] = FindSet(parent_[v]);
        return parent_[v];
    }

    void Union(const Vertex& u, const Vertex& v) {
        Vertex a = FindSet(u);
        Vertex b = FindSet(v);
        if (a != b)
            Link(a, b);
    }
};

// Алгоритм Краскала: минимальное остовное дерево.
Cost Kruskal(Edges& edges, Edges& mst) {
    Cost result = 0;
    mst.clear();

    if (edges.empty())
        return result;

    Vertex count = 0;
    for (const Edge& e : edges)
        count = max(count, max(e.from, e.to) + 1);

    DisjoinSets sets(count);
    sort(edges.begin(), edges.end(), [](const Edge& a, const Edge& b) {
        return tie(a.cost, a.from, a.to) < tie(b.cost, b.from, b.to);
    });

    for (const Edge& e : edges) {
        if (sets.FindSet(e.from) != sets.FindSet(e.to)) {
            sets.Union(e.from, e.to);
            mst.push_back(e);
            result += e.cost;
            if (mst.size() == static_cast<size_t>(count - 1))
                break;
        }
    }
    return result;
}

// DOT: рёбра остова выделяются красным и толщиной.
void PrintDot(ostream& os, const Edges& edges, const Edges& mst) {
    set<Edge> mst_edges(mst.begin(), mst.end());
    os << "graph G {\n";
```

```
    for (const Edge& e : edges) {
        os << "  " << e.from << " -- " << e.to
           << " [label=\"" << e.cost << "\"";
        if (mst_edges.count(e) > 0)
            os << ", color=red, penwidth=3";
        os << "];\n";
    }
    os << "}\n";
}

int main() {
    Edges edges = {
        {0, 1, 1}, {0, 2, 7}, {0, 3, 8},
        {0, 4, 6}, {1, 4, 1}, {2, 1, 4},
        {2, 5, 4}, {3, 5, 12}, {4, 5, 8}
    };
    Edges mst;

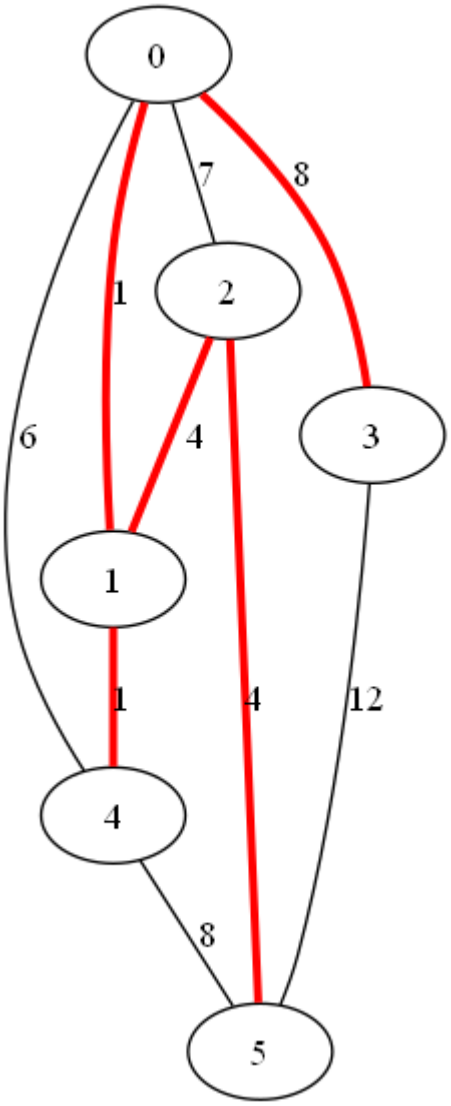
    Cost mstCost = Kruskal(edges, mst);

    ofstream file("graph.dot");
    if (!file) {
        cerr << "Cannot rewrite graph.dot file\n";
        return 1;
    }
    PrintDot(file, edges, mst);
    file.close();

    cout << "MST cost: " << mstCost << '\n';
    if (system("dot -Tpng graph.dot -o graph.png") == 0)
        cout << "Graphviz: graph.png created.\n";
    else
        cerr << "Could't run dot. Check PATH.\n";

    return 0;
}
```

Граф из примера:



Алгоритм Краскала работает в следующем порядке:

1. Получает вектор ребер и считает количество вершин.
2. Создает вектор вершин и заполняет `parent_` и `rank_` нулями. Что означает что все вершины по умолчанию корни. Лес из деревьев длины 1.
3. Сортирует ребра в порядке возростания по приоритету cost, from, to. (можно было только по cost).
4. Проходит отсортированные ребра и в случае несовпадения e.from e.to объединяет соответствующие вершины в векторе вершин.
5. Кладет в вектор остовного дерева соотетствующее ребро e. И увеличиваем стоимость result на соответст. cost.
6. Выполняем до тех пор, пока число ребер не будет соотвестсовать числу вершин - 1.